

Space-based Parameter Evolving with Lightweight Optimization for Graph Adaptation to Evolving Shifts

Junyu Luo*
Peking University
Beijing, China
luojunyu@stu.pku.edu.cn

Hourun Li*
Peking University
Beijing, China
2301213258@stu.pku.edu.cn

Zixuan Ouyang*
Peking University
Beijing, China
1900013083@pku.edu.cn

Zhiping Xiao†
University of Washington
Seattle, USA
patxiao@uw.edu

Ming Zhang*†
Peking University
Beijing, China
mzhang_cs@pku.edu.cn

Xiao Luo†
University of Wisconsin–Madison
Madison, USA
xiao.luo@wisc.edu

Yifan Wang
University of International Business
and Economics
Beijing, China
yifanwang@uibe.edu.cn

Abstract

Adapting graph neural networks to evolving domain shifts presents a fundamental challenge: how to acquire new knowledge while preventing catastrophic forgetting. Existing continual learning methods often rely on memory replay or complex regularization schemes, incurring significant computational overhead. We propose STEM (Space-based Parameter Evolving with Lightweight Optimization), a replay-free framework that transforms continual adaptation into controlled parameter space evolution via a controller-worker architecture. At its core is a Test-time Evolving State Space (TESS) controller with a selective gating mechanism that recursively updates its hidden state by integrating compact summaries of the current graph domain. Unlike traditional linear state space models, TESS enables nonlinear, input-dependent state transitions that capture the temporal dynamics of domain evolution. A lightweight parameter generator decodes this evolving state into domain-specific adapter parameters injected into a frozen base GNN. We employ unsupervised Information Maximization and parameter space stability regularization that penalizes adapter changes across time steps, with theoretical guarantees of forgetting mitigation and stable convergent adaptation. Extensive experiments validate that our method achieves state-of-the-art performance while maintaining minimal computational overhead. The code is available at <https://github.com/miaomiao1220/stem>.

*National Key Laboratory for Multimedia Information Processing, School of Computer Science, Beijing Key Laboratory of Software and Hardware Cooperative Artificial Intelligence Systems, Peking University.

†Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792714>

CCS Concepts

• **Computing methodologies** → **Neural networks; Continual learning**; *Unsupervised learning*.

Keywords

Graph Adaptation to Evolving Shifts, Graph Continual Learning, Graph Domain Adaptation, Parameter-Efficient Fine-Tuning, Catastrophic Forgetting, State Space Model, Unsupervised Learning

ACM Reference Format:

Junyu Luo, Zixuan Ouyang, Xiao Luo, Hourun Li, Zhiping Xiao, Yifan Wang, and Ming Zhang. 2026. Space-based Parameter Evolving with Lightweight Optimization for Graph Adaptation to Evolving Shifts. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792714>

1 Introduction

Graph-structured data plays a fundamental role in modeling real-world systems, and the development of deep learning-based graph modeling methods has been successfully applied to numerous domains, including recommender systems [11, 41], bioinformatics [24, 38], RAG [44], and social networks [5, 45]. Graph neural networks have demonstrated superior performance on static graph-structured data [18, 35]. However, in the real world, graph data often arrives in a sequential manner, with its underlying distribution continuously evolving over time or space, forming a series of dynamic domains. When a GNN model trained on a source domain is deployed to such a sequence of evolving target domains, it faces two critical challenges: poor performance on new domains due to insufficient adaptability, and catastrophic forgetting of knowledge learned on old domains after adapting to new ones [22, 32].

Existing graph continual learning and domain adaptation methods primarily rely on memory replay, regularization, or parameter isolation. Memory replay methods [32] consolidate old knowledge

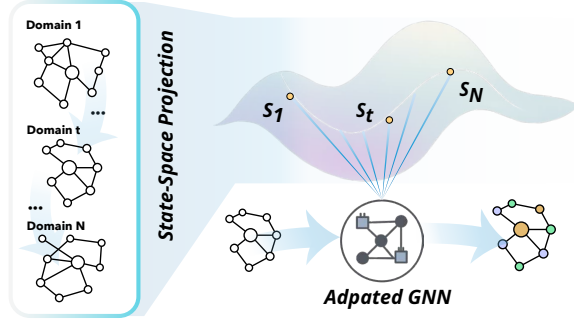


Figure 1: Framework overview. The controller projects evolving graph domains (top) into a state sequence S_t that captures historical dynamics (middle). Each state generates domain-specific adapters to adjust the shared GNN (bottom).

by storing and replaying historical data or its compressed representations, but they incur significant storage and computational overhead. Regularization methods [21, 43] mitigate forgetting by penalizing modifications to important parameters, but they may over-constrain the model and sacrifice its adaptability. Parameter isolation methods [29, 46, 48] allocate new parameters for new tasks, but this leads to continuous growth in model size.

To overcome these limitations, we propose **Space-based Parameter Evolving with Lightweight Optimization (STEM)**, a novel continual adaptation framework for evolving graph domains, as illustrated in Figure 1. This framework requires no replay of historical data, and its core is a controller-worker architecture. We maintain a base GNN with frozen parameters as the *worker*, and design a lightweight *controller* driven by a Test-time Evolving State Space (TESS) module. Unlike traditional linear state space models, TESS employs a selective gating mechanism to enable nonlinear, input-dependent state transitions. At each time step, the controller receives a compact summary of the current graph domain and updates its internal hidden state through this gating mechanism, which encapsulates the entire evolution history of the model up to the current moment. Subsequently, a parameter generator transforms the state into a set of adapter parameters, which are injected into the *worker* GNN to achieve domain-specific adaptation at minimal cost. The key innovation lies in reframing continual graph learning from a temporal control perspective: instead of treating each domain adaptation as an isolated optimization task, we view the entire adaptation trajectory as a dynamical system evolving in low-dimensional parameter space, enabling TESS to learn forward-looking, smooth transitions between domains rather than reactive adjustments.

In order to enable effective unsupervised adaptation, we design an unsupervised Information Maximization objective to encourage confident yet diverse predictions. We introduce a parameter space stability regularization to mitigate catastrophic forgetting. The regularization directly penalizes the Frobenius norm of adapter parameter changes across adaptation steps, ensuring smooth model evolution and provably bounding output drift on historical domains.

Our framework transforms the problem of continual adaptation on graphs into one of temporal modeling and control of the model’s parameter evolution dynamics, achieving an efficient, lightweight, and replay-free solution. The main contributions are: ❶ We propose STEM, featuring a controller-worker architecture with a Test-time

Evolving State Space (TESS) module that employs gating-based nonlinear state transitions to model temporal dynamics of domain evolution, enabling parameter-efficient continual adaptation by generating lightweight adapters from evolving states rather than storing historical data. ❷ We introduce a training framework combining unsupervised Information Maximization for adaptation with parameter space stability regularization, and provide rigorous theoretical analysis demonstrating that our approach provably bounds catastrophic forgetting and ensures convergent adaptation. ❸ Extensive experiments validate that STEM achieves state-of-the-art performance in handling continuous graph domain shifts while maintaining minimal computational and memory overhead.

2 Related Work

Graph Continual Learning. Graph continual learning (GCL) aims to enable graph models to sequentially acquire new knowledge while avoiding catastrophic forgetting of previously learned information. Existing approaches can be roughly divided into replay-based, regularization-based, and parameter isolation methods. Replay-based methods [23, 32, 47, 49, 50, 52] consolidate old knowledge by storing and replaying historical data or compressed representations, incurring significant storage and computational overhead. Regularization-based methods [1, 21, 34, 43] mitigate forgetting by penalizing modifications to important parameters, but may over-constrain the model and sacrifice adaptability to new domains. Parameter isolation methods [29, 46, 48] allocate dedicated parameters for new tasks, leading to continuous model growth. In contrast, our framework introduces a replay-free paradigm that fundamentally decouples knowledge storage from dynamic adaptation through a controller-worker architecture, reframing continual learning as temporal modeling of parameter evolution dynamics.

Test-Time Adaptation on Graphs. To extend graph domain adaptation [3, 22, 28, 39, 42] to the online setting, Graph test-time adaptation (TTA) and its continual variant (CTTA) focus on online model adjustment using only unlabeled target data streams after deployment. Existing CTTA methods [20, 27, 30, 36, 37] typically rely on unsupervised objectives such as entropy minimization [6] and pseudo-label self-training [2, 4, 25, 26]. However, these approaches often passively and heuristically resist forgetting without explicitly modeling the domain evolution process, making them prone to error accumulation and model drift under drastic domain shifts. Our Test-time Evolving State Space (TESS) module represents a paradigm shift by introducing state space modeling to this problem, actively and explicitly capturing temporal dependencies across domains to predict long-term adaptation dynamics.

Parameter-Efficient Fine-Tuning. Parameter-efficient fine-tuning (PEFT) techniques [10], including LoRA [13], Adapter [12], and Prefix-Tuning [19], were originally designed to efficiently transfer large pre-trained networks to downstream scenarios by tuning a limited number of parameters. Our core contribution fundamentally differs from traditional PEFT research. Existing PEFT work [8, 51] primarily addresses single, static downstream task adaptation, whereas our work tackles dynamic, continuous domain evolution. Our innovation lies not in the adapter itself, but in designing an evolution controller that dynamically generates adapter parameters, transforming PEFT into a self-adjusting system that

responds to environmental changes, thereby significantly expanding its applicability to online learning and continual adaptation scenarios.

3 Methodology

3.1 Preliminaries and Overview

Our framework addresses the problem of continual, unsupervised domain adaptation over streaming graph data. Given a base GNN f_{Θ_0} pre-trained on a source domain (or a collection of source graphs), we consider a temporal sequence of graph domains $\{G_1, G_2, \dots, G_T\}$ arriving in chronological order, where $G_t = (\mathcal{V}_t, \mathcal{E}_t, X_t)$ represents the graph structure and node features of the t -th domain. Our objective is to construct a working model at each time step t by injecting lightweight adapter parameters without modifying the base parameters Θ_0 as $f_t(x) \equiv f_{\Theta_0 \oplus \mathcal{A}(v_t)}(x)$, where v_t denotes the adapter vector produced by our controller and generator, $\mathcal{A}(\cdot)$ represents the operation that maps this vector into pluggable modules (e.g., LoRA, bottleneck adapters, or bias tuning) for each layer, and $\oplus$ denotes the injection into the base network.

The core philosophy of our framework (illustrated in Figure 2) is to decouple the model’s knowledge base from its adaptation mechanism. We employ a base GNN f_{GNN} whose main parameters Θ_{base} remain frozen throughout the adaptation process. The model’s evolution is entirely realized through a series of dynamically generated, lightweight adapter parameters $\{\Delta\Theta_1, \Delta\Theta_2, \dots, \Delta\Theta_T\}$. The key insight of this design is that we transform the complex continual learning problem into a simpler, more focused task: learning a temporal model that can predict the optimal adapter parameters $\Delta\Theta_t$ based on the historical evolution state and current domain information. This temporal model is jointly implemented by our Test-time Evolving State Space controller and the parameter generator.

3.2 Evolving Parameter Space via State-Guided Lightweight Adapters

Continual adaptation over evolving graph domains presents a fundamental dilemma: how can a model simultaneously adapt to new distributions and preserve past knowledge while maintaining computational efficiency? Fine-tuning entire GNNs is prohibitively expensive and treats adaptation as isolated optimization problems, leading inevitably to catastrophic forgetting. We propose a paradigm shift: instead of manipulating high-dimensional weight spaces, we transform adaptation into controlled evolution within a low-dimensional, dynamically generated adapter parameter space.

Lightweight Adaptation via Functional Bottleneck Parameterization. We instantiate the adapter $\Delta\Theta_t$ using functional bottleneck adapters, which operate in the activation space rather than directly modifying weight matrices. For each layer l in the Worker GNN, we inject a lightweight residual transformation applied to the activations. At time step t , the adapted activation becomes:

$$h_{out}^{(l)} = h_{in}^{(l)} + \mathcal{A}_t^{(l)}(h_{in}^{(l)}; \Delta\Theta_t^{(l)}), \quad (1)$$

where $h_{in}^{(l)}$ is the input activation, and $\mathcal{A}_t^{(l)}(\cdot; \Delta\Theta_t^{(l)})$ denotes a parametric transformation (e.g., a bottleneck MLP with down-projection, nonlinearity, and up-projection) controlled by adapter parameters $\Delta\Theta_t^{(l)}$. The bottleneck dimension is significantly smaller than the

layer’s hidden dimension, ensuring that $|\Delta\Theta_t^{(l)}| \ll |\Theta_{base}^{(l)}|$. The complete adapter $\Delta\Theta_t = \{\Delta\Theta_t^{(l)}\}_{l=1}^L$ encompasses all layers. This design reduces the parameter space from millions of base parameters to thousands of adapter parameters, enabling efficient adaptation.

From Static Adaptation to Dynamic Evolution. While lightweight adapters enable efficient parameter injection, a question emerges: how should $\Delta\Theta_t$ be dynamically generated at each time step? Naively optimizing from scratch on each domain leads to oscillations and catastrophic forgetting, as each adaptation ignores accumulated wisdom. The crux is not merely updating parameters lightly, but orchestrating their evolution intelligently across time.

We reframe this as meta-control: rather than learning task-specific knowledge, we learn a temporal controller governing adapter evolution based on historical states and current domain characteristics. This is the role of our Test-time Evolving State Space (TESS)—a meta-controller that learns how to learn. TESS navigates the adapter parameter space, producing at each step the optimal $\Delta\Theta_t$ that balances current adaptation with past preservation, directly embodying our title’s core philosophy: Space-based Parameter Evolving.

Graph-Driven State Evolution. TESS evolution is fundamentally data-driven, grounded in current graph characteristics. We establish a perceptual bridge between graph domain G_t and TESS through a domain summary $\tilde{h}_t \in \mathbb{R}^{d_h}$ —a fixed-dimensional representation encapsulating distributional properties of G_t , extracted via lightweight encoding and readout using the frozen base GNN:

$$\tilde{h}_t = \text{READOUT}(\{h_i^{(L')}\}) = \text{READOUT}(\{f_{GNN}^{(L')}(G_t; \Theta_{base})_i\}_{i \in \mathcal{V}_t}), \quad (2)$$

where READOUT (e.g., mean pooling or attention) produces a compact graph representation. This summary captures the base model’s perception under its current state, and its divergence from historical representations reflects domain drift magnitude.

This design establishes the Graph-based nature of our framework. Adapter parameters $\Delta\Theta_t$ are not generated in isolation, but result from TESS’s deliberate decision-making after observing current graph characteristics ($\tilde{h}_t$) and synthesizing them with accumulated memory (S_{t-1}). The adaptation is simultaneously graph-aware, history-aware, and context-sensitive.

The Complete Framework. The full workflow forms a closed loop: ❶ Graph domain G_t is perceived and distilled into domain summary $\tilde{h}_t$ via the frozen base GNN; ❷ TESS receives $\tilde{h}_t$ and previous state S_{t-1} , computing new evolution state S_t —the model’s current coordinate on its parameter evolution trajectory; ❸ Parameter generator g_{gen} decodes this low-dimensional state S_t into concrete adapter parameters $\Delta\Theta_t$; ❹ Adapters are injected into the Worker GNN for inference on G_t . This architecture decouples perception, decision, generation, and execution, forming a hierarchical control system for continual graph learning.

3.3 Test-time Evolving State Space and Parameter Generator

Having established TESS as the meta-controller, we now detail its internal mechanism—a dynamic system embodying intelligent parameter space navigation. At each step t , TESS receives previous evolution state $S_{t-1} \in \mathbb{R}^{d_s}$ and current domain summary $\tilde{h}_t \in \mathbb{R}^{d_h}$,

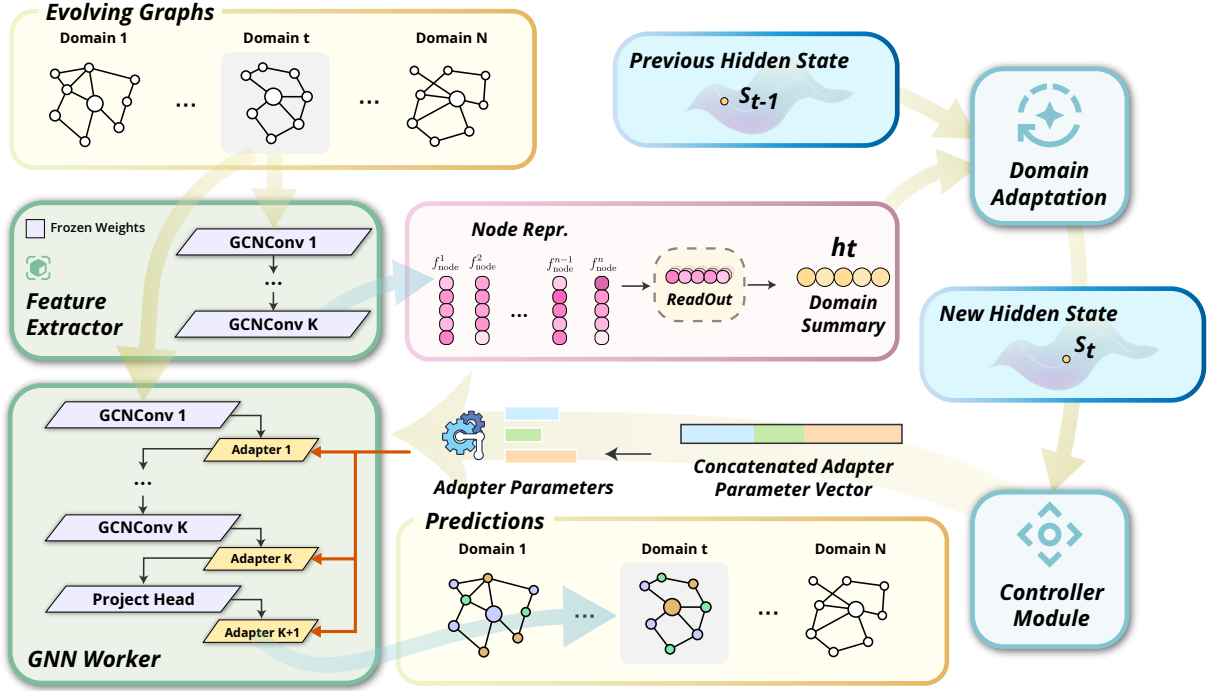


Figure 2: Overview of STEM. The framework employs a Test-time Evolving State Space (TESS) controller (upper right) to dynamically generate adapter parameters, which are injected into the Worker GNN (lower left) to enable efficient adaptation across continuously evolving graph domains (upper left) without storing or replaying historical data.

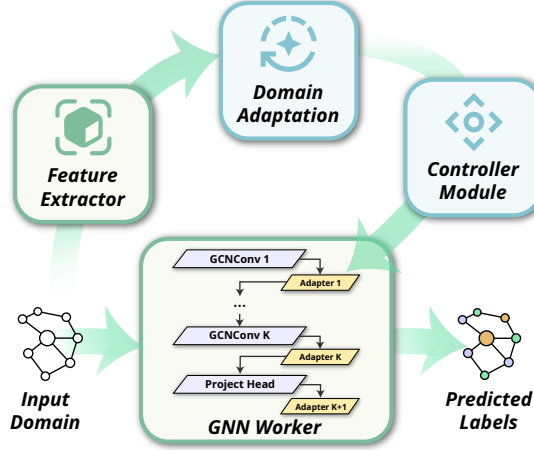


Figure 3: Architecture of the Test-time Evolving State Space controller with gating mechanism for state evolution.

computing new evolution state $S_t \in \mathbb{R}^{d_s}$ representing the model's updated position on its parameter evolution trajectory (Figure 3). **Why Linear State Space Models Fall Short.** Traditional SSMs adopt linear transitions $s_t = As_{t-1} + Bu_t$ with fixed matrices (A, B) . While elegant for physical systems, this proves inadequate for GNN adaptation facing complex domain shifts—an inherently nonlinear process where small distributional changes can necessitate disproportionate parameter adjustments. Our TESS is a nonlinear dynamical system with input-dependent transitions. Rather than rigid

transformations, TESS employs learned gating dynamically modulated by current domain observation $\bar{h}_t$, enabling context-sensitive decisions about evolution magnitude and direction.

Selective Gating Mechanism. TESS balances preserving historical knowledge and adapting to new domains through a gating mechanism controlled by the current domain summary $\bar{h}_t$, which dynamically regulates information flow and adaptation decisions.

The mechanism employs three gates computed from $\bar{h}_t$. The **forget gate** f_t determines which aspects of historical state to discard:

$$f_t = \sigma(W_f \bar{h}_t + b_f), \quad (3)$$

The **update gate** u_t controls the magnitude of new information:

$$u_t = \sigma(W_u \bar{h}_t + b_u), \quad (4)$$

The **write scale** α_t provides bidirectional modulation:

$$\alpha_t = \tanh(W_\alpha \bar{h}_t + b_\alpha). \quad (5)$$

Next, a candidate state update is generated by synthesizing current observation with historical context:

$$\tilde{S}_t = \tanh(W_{mix} [\bar{h}_t; S_{t-1}] + b_{mix}), \quad (6)$$

where $[\cdot; \cdot]$ denotes concatenation. Finally, the new state is formed through additive composition with layer normalization:

$$S_t = \text{LayerNorm} \left(\underbrace{(1 - f_t) \odot S_{t-1}}_{\text{Retained Memory}} + \underbrace{u_t \odot (\alpha_t \odot \tilde{S}_t)}_{\text{Scaled Innovation}} \right). \quad (7)$$

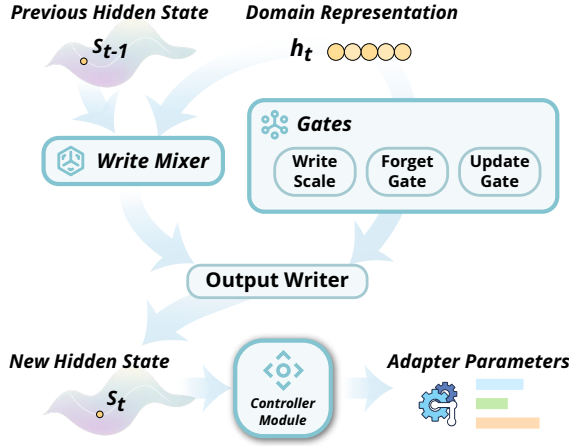


Figure 4: Detailed mechanism of state evolution and adapter parameter generation in the TESS module. Three gates (Forget, Update, Write Scale) are computed from domain summary h_t to control state transition dynamics.

When domain shift is mild, f_t remains low and u_t is modest; when shift is severe, gates adjust accordingly. The write scale α_t can amplify or suppress updates. Critically, this gating policy is learned from data, allowing TESS to autonomously discover optimal adaptation tempo for each transition.

From Evolution State to Adapter Parameters. Evolution state S_t encodes the optimal position in abstract parameter evolution space. To modify the Worker GNN, this abstract state must be decoded into concrete adapter parameters via parameter generator g_{gen} :

$$\text{vec}(\Delta\Theta_t) = g_{gen}(S_t; \Theta_{gen}), \quad (8)$$

where $\text{vec}(\Delta\Theta_t)$ is the vectorized adapter parameters $\{\Delta\Theta_t^{(l)}\}_{l=1}^L$ for all layers, and g_{gen} is a multi-layer perceptron. This enables crucial knowledge compression: rather than storing adapter parameters for every domain (growing linearly with T), we store only constant-size state S_t and learn a mapping to synthesize appropriate adapters on-the-fly. This ensures S_t guides the generator to optimally balance accumulated knowledge with current requirements. The complete workflow is illustrated in Figure 4.

3.4 Training Objectives

Our training procedure jointly optimizes two complementary objectives that enable effective adaptation while preventing catastrophic forgetting.

Unsupervised Task Loss. Since the target domain has no labels, we adopt an unsupervised loss based on Information Maximization. This loss encourages our model to produce high-confidence (low-entropy) predictions while ensuring that the predicted class distribution across the entire graph exhibits diversity (high entropy), thereby preventing the model from collapsing to a trivial solution. Let $p_v = \text{Softmax}(f(G_t; \Theta_t)_v)$ denote the model's output probability distribution for node v , and $\bar{p} = \mathbb{E}_{v \in \mathcal{V}_t} [p_v]$ denote the average probability distribution over the graph. The task loss is

defined as:

$$\mathcal{L}_{\text{task}}(G_t; \Theta_t) = \underbrace{-\mathbb{E}_{v \sim \mathcal{V}_t} \left[\sum_{k=1}^C p_{v,k} \log(p_{v,k}) \right]}_{\text{Minimize Node Entropy (Increase Confidence)}} + \underbrace{\sum_{k=1}^C \bar{p}_k \log \bar{p}_k}_{\text{Maximize Global Entropy (Increase Diversity)}}, \quad (9)$$

Parameter Space Stability Loss. This is the core regularization term we design to ensure smooth model evolution. We directly penalize the divergence between adapter parameters produced by the parameter generator at consecutive time steps $t-1$ and t . The loss is measured using the squared Frobenius norm over all adapter parameters across all layers:

$$\mathcal{L}_{\text{stab}} = \|\Delta\Theta_t - \Delta\Theta_{t-1}\|_F^2, \quad (10)$$

where $\Delta\Theta_t = \{\Delta\Theta_t^{(l)}\}_{l=1}^L$ comprises all adapter parameters (including projection weights, biases, and scaling factors) across all layers. This loss term forces the model's adaptation actions to be continuous and gradual, effectively preventing drastic parameter oscillations caused by adapting to new domains, thereby fundamentally preserving historical information at the parameter level.

3.5 Optimization and Training Flow

At each time step t , we jointly optimize the two loss terms above. The total loss function is:

$$\min_{\Theta_{\text{TESS}}, \Theta_{\text{gen}}} \mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_{\text{stab}} \mathcal{L}_{\text{stab}}, \quad (11)$$

where λ_{stab} is to balance the loss terms, which is set to 0.1 according to Section 4.3. By optimizing this objective function via gradient descent, we update only the TESS controller and parameter generator's parameters, while the Worker GNN's base parameters remain frozen. This process enables our framework to learn a meta-level adaptation strategy to handle continuous domain evolution.

Exponential Moving Average for Stable Inference. To further enhance the stability and robustness of our framework during continual adaptation, we maintain an Exponential Moving Average (EMA) of the controller parameters $(\Theta_{\text{TESS}}, \Theta_{\text{gen}})$. The EMA model serves as a slowly-updated shadow version of the actively trained controller, providing more stable predictions by smoothing out transient fluctuations during gradient-based adaptation. At each optimization step within domain t , after updating the controller parameters via gradient descent, we update the EMA parameters:

$$\Theta_{\text{EMA}} \leftarrow \alpha \cdot \Theta_{\text{EMA}} + (1 - \alpha) \cdot \Theta_{\text{current}}, \quad (12)$$

where $\Theta_{\text{current}} = (\Theta_{\text{TESS}}, \Theta_{\text{gen}})$ denotes the current controller parameters after gradient update, Θ_{EMA} represents the EMA version, and α is the momentum coefficient controlling the averaging rate, which is set to 0.9 according to Section 4.3. Note that since the Worker GNN's base parameters Θ_{base} remain frozen throughout training, the EMA only tracks the evolution of the trainable controller components. For final inference and evaluation, we use the EMA controller to generate adapter parameters.

3.6 Theoretical Analysis

Having introduced the two loss components and their joint optimization, we now provide theoretical insights into why this design

provably mitigates catastrophic forgetting and ensures stable adaptation. We present two theorems that formalize the guarantees offered by our parameter space stability regularization $\mathcal{L}_{\text{stab}}$. The detailed proofs are provided in the Appendix.

THEOREM 3.1 (BOUND ON CATASTROPHIC FORGETTING). *Assume the Worker GNN’s output function $f(G; \Theta_{\text{base}} \oplus \Delta\Theta)$ is L -Lipschitz continuous with respect to its adapter parameters $\Delta\Theta$. Then, when transitioning from domain $t-1$ to domain t , the expected change in the model’s outputs on all historical graph domains $G_{<t} = \{G_1, \dots, G_{t-1}\}$ is bounded by the parameter evolution stability loss:*

$$\mathbb{E}_{G \sim G_{<t}} [\|f(G; \Theta_{\text{base}} \oplus \Delta\Theta_t) - f(G; \Theta_{\text{base}} \oplus \Delta\Theta_{t-1})\|_2] \leq L \cdot \|\Delta\Theta_t - \Delta\Theta_{t-1}\|_F = L \cdot \sqrt{\mathcal{L}_{\text{stab}}}, \quad (13)$$

where L is the Lipschitz constant and $\|\cdot\|_F$ is the Frobenius norm.

This theorem quantifies how our model resists forgetting from a theoretical perspective. Catastrophic forgetting can be mathematically characterized as a drastic change in the model’s outputs on old data after training on new data. Theorem 3.1 establishes that the magnitude of output changes on historical domains is directly controlled by the change in adapter parameters. By explicitly penalizing $\mathcal{L}_{\text{stab}}$ during training, we enforce that consecutive adapter updates remain small in parameter space, which guarantees minimal output perturbations on past domains. This provides a formal justification for using parameter space regularization as a proxy for preventing functional drift on historical data without requiring explicit storage or replay of past samples.

THEOREM 3.2 (CONVERGENCE AND STABILITY OF THE ADAPTATION PROCESS). *Assume that for any domain t , the task loss $\mathcal{L}_{\text{task}}$ is convex with respect to the controller parameters $(\Theta_{\text{TESS}}, \Theta_{\text{gen}})$ and has bounded gradients. Under the regularization imposed by the parameter evolution stability loss $\mathcal{L}_{\text{stab}}$, the total loss function $\mathcal{L}_{\text{total}}$ becomes locally strongly convex near a minimizer under standard assumptions. Consequently, the adaptation optimization process on each domain converges to a unique optimal controller parameter, and this solution is stable. Formally, when the Jacobian of the adapter generation map has full rank and the residual is sufficiently small, the Hessian of the total loss satisfies:*

$$\nabla^2 \mathcal{L}_{\text{total}} = \underbrace{\nabla^2 \mathcal{L}_{\text{task}}}_{\text{Positive Semidefinite}} + \underbrace{\lambda_{\text{stab}} \nabla^2 \mathcal{L}_{\text{stab}}}_{\text{Locally Positive Definite}} \succ 0, \quad (14)$$

implying that the optimization problem is locally strongly convex near the solution.

This theorem guarantees that our adaptation process is reliable and well-behaved. While $\mathcal{L}_{\text{task}}$ drives adaptation to new domains, the stability loss $\mathcal{L}_{\text{stab}}$ plays a crucial dual role: it suppresses forgetting (Theorem 3.1) and induces local strong convexity near the solution, ensuring unique local solutions and stable convergence. Importantly, by keeping adapter parameter changes small, $\mathcal{L}_{\text{stab}}$ maintains the conditions (small residuals, full-rank Jacobian) required for local strong convexity throughout continual adaptation. Together, these two theorems demonstrate that our framework achieves both stability and adaptability through principled parameter space regularization.

3.7 Complexity Analysis

The computational overhead of our algorithm for adaptation on each test-time domain is extremely low. The primary computational complexity stems from an optimization step on the current graph G_t , used for extracting the domain summary and computing the task loss, with complexity approximately $O(|\mathcal{E}_t|d^2)$, where $|\mathcal{E}_t|$ is the number of edges and d is the node feature dimension. Beyond this, the computational complexity of the TESS controller and parameter generator depends only on their own small, fixed number of parameters, and is independent of the input graph’s scale (number of nodes $|V_t|$ and edges $|\mathcal{E}_t|$), making their overhead negligible. Compared to methods requiring storage and replay of historical data, our algorithm’s space complexity is also extremely low, requiring only storage of a low-dimensional evolution state vector S_t and the controller’s small number of parameters. Thus, this framework achieves efficient online continual adaptation with minimal computational and storage overhead.

4 Experiment

4.1 Experimental Settings

Datasets. Our empirical evaluation is conducted on four widely adopted graph benchmarks, specifically curated to simulate continuously evolving domains and including both node-level and graph-level classification tasks. For node-level classification, we utilize Twitch-explicit [33] to simulate regional distribution shifts across different social networks and OGB-Arxiv [14] to model temporal evolution through its chronologically partitioned citation networks. For graph-level classification, we employ Mutagenicity [16] and FRANKENSTEIN [31], two bioinformatics datasets comprising molecular and protein structures. To simulate a continuous evolution of structural properties, we sequence the graphs in these datasets based on their graph density.

Baselines. We compare our method against state-of-the-art approaches spanning three categories. Domain Adaptation methods include DANN [7] and GTrans [15], which adapt models across graph domains. Test-Time Adaptation methods include Norm [20], TENT [36], and EATA [30], which perform unsupervised adaptation at test time without retraining. Continual Learning methods include EERM [40], CoTTA [37], and GCAL [32], which are specifically designed to handle sequential domain shifts while mitigating catastrophic forgetting. Details are provided in Appendix B.1.

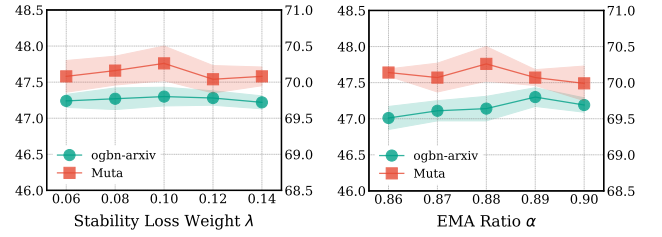
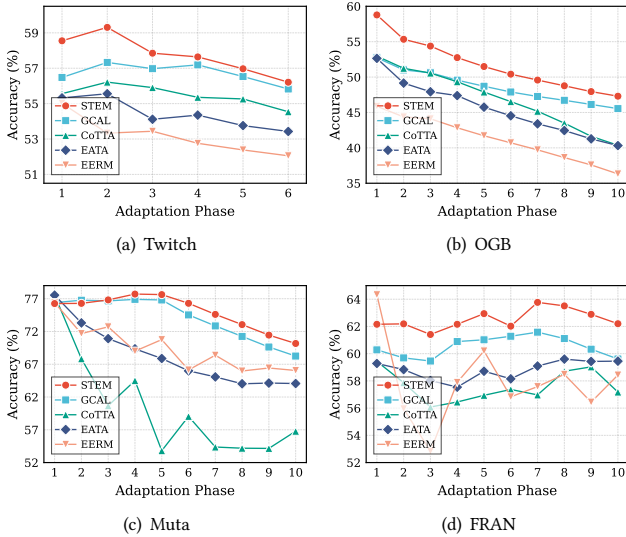
Evaluation Metrics. Our evaluation employs a performance matrix $M^{\text{result}} \in \mathbb{R}^{T \times T}$ where $M_{i,j}^{\text{result}}$ (for $i \geq j$) records performance on domain j after training on domain i . We derive two complementary metrics to assess catastrophic forgetting: Average Performance (AP) as $\frac{1}{T} \sum_{i=1}^T M_{T,i}^{\text{result}}$, which evaluates the final model’s retention across all historical domains (the last row of the matrix), and Evolution Average Performance (EAP) as $\frac{1}{T} \sum_{i=1}^T M_{i,i}^{\text{result}}$, which captures immediate adaptation quality on each domain (the diagonal elements). If a model suffers from severe catastrophic forgetting, its performance on old domains degrades after adapting to new ones, resulting in lower AP. The gap between EAP and AP quantifies knowledge degradation throughout the continual learning process: a smaller gap indicates better knowledge preservation, while a larger gap reveals more severe forgetting as adaptation progresses.

Table 1: Data performance (%) comparison across four datasets, evaluated against eight baseline methods and STEM.

Methods	Twitch-explicit		OGB-Arxiv		Mutagenicity		FRANKENSTEIN		Avg	
	AP↑	EAP↑	AP↑	EAP↑	AP↑	EAP↑	AP↑	EAP↑	AP↑	EAP↑
DANN	51.50±0.39	51.31±0.67	42.62±0.23	43.90±0.28	68.14±0.14	64.56±0.45	58.87±1.34	56.06±1.63	55.28	53.96
Norm	52.65±0.00	54.95±0.00	40.21±0.00	40.10±0.00	66.26±0.40	63.46±0.93	57.50±1.18	55.92±1.30	54.16	53.61
TENT	52.84±0.98	54.67±1.44	36.72±0.37	36.74±0.41	64.93±0.03	65.03±0.03	59.56±0.08	57.60±0.09	53.51	53.51
EERM	52.08±0.00	52.08±0.00	36.26±0.00	36.26±0.00	65.89±0.17	64.88±0.70	58.40±0.09	57.19±0.12	53.16	52.60
GTrans	53.55±0.29	53.55±0.29	39.69±0.04	39.69±0.04	68.83±0.45	65.60±0.32	58.07±0.01	56.48±0.05	55.03	53.83
CoTTA	53.94±0.36	53.60±0.55	40.28±0.01	42.24±0.01	56.74±0.04	63.06±0.04	58.36±0.70	55.87±1.66	52.33	53.69
EATA	53.56±0.10	54.28±0.12	40.91±0.70	42.26±0.92	64.11±0.07	64.97±0.10	59.48±0.06	57.71±0.09	54.52	54.80
GCAL	55.65±0.09	55.23±0.16	45.22±0.17	44.46±0.18	68.14±0.14	68.35±1.07	59.66±0.02	58.21±0.04	57.17	56.56
STEM	56.16±0.04	56.19±0.09	47.30±0.16	47.47±0.18	70.26±0.24	69.50±0.40	61.54±0.71	60.02±0.72	58.81	58.30

Table 2: Ablation studies on Twitch-explicit and OGB-Arxiv datasets. M1-M5 are the different versions of STEM.

Model	EMA	Stability Loss	TESS	Twitch-explicit	OGB-Arxiv
				AP↑	AP↑
M1				54.56±0.02	36.68±2.05
M2		✓		55.67±0.01	46.81±0.01
M3	✓			53.49±0.01	46.50±0.44
M4			✓	54.55±0.04	34.81±0.01
M5	✓		✓	52.66±0.20	34.24±0.53
Full	✓	✓	✓	56.16±0.04	47.30±0.16

**Figure 6: Sensitivity analysis on hyperparameters.****Figure 5: Performance evolution throughout continuous adaptation phases on evolving distribution-shifted graphs.**

4.2 Results

4.2.1 Main Results. We first examine the overall performance. As shown in Table 1, STEM achieves state-of-the-art performance on all datasets, setting new best results in both AP and EAP during the continuous adaptation phase. While GCAL performs well, STEM

exhibits a significant advantage due to its fundamental architectural differences. GCAL relies on regularization to passively constrain forgetting, while STEM’s TESS controller actively captures the temporal dynamics of domain evolution through a nonlinear state-space mechanism. By combining historical evolutionary states with current graph domain features, TESS predicts and generates optimal adapter parameters at each time step, thus achieving a forward-looking, smooth adaptation strategy rather than passive adjustment. As for test-time adaptive methods such as TENT and EATA, they suffer from a key limitation. Due to a lack of memory, they cannot accumulate knowledge during continuous domain transformations, leading to performance degradation when facing continuous distribution drift.

4.2.2 Adaptation Dynamics. We designed experiments to examine dynamic adaptation capabilities, as shown in Figure 5. We primarily focus on the change in mean accuracy over previously encountered domains after adapting to a new domain. Most methods exhibited performance degradation during adaptation, confirming that catastrophic forgetting is a core challenge in continuous graph learning. Baseline methods showed a sharp performance drop after several adaptation phases, indicating instability in their mechanisms. The adaptation methods showed even greater performance degradation during testing. STEM maintained consistently superior performance across all datasets and exhibited the slowest performance degradation, thanks to the TESS controller’s temporal modeling of domain evolution dynamics, demonstrating its excellent resistance to forgetting. Figure 7 provides a fine-grained view by

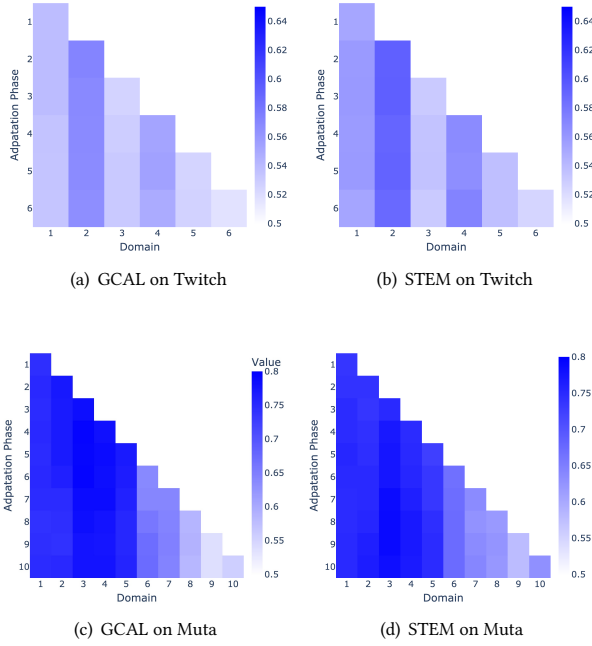


Figure 7: Per-domain accuracy heatmaps throughout the continual adaptation process. Rows represent adaptation phases, and columns are test domains. Darker for higher accuracy.

domain, showing that STEM maintains more stable performance over historical domains compared to the robust baseline methods.

4.3 Analysis

4.3.1 Ablation Study. We designed ablation experiments to validate the effectiveness of each core component of our method. We primarily examined three key elements: the TESS controller, the use of exponential moving average (EMA), and the parameter space stability loss ($\mathcal{L}_{stab}$). The results are shown in Table 2. During adaptation to the new domain, updating the controller parameters with EMA provides a more stable and robust model for the final evaluation. Our ablation experiments investigated the impact of this technique. The complete method consistently maintained optimal performance, while removing any component resulted in performance degradation. Further analysis revealed key synergies between components. Notably, when the TESS controller was used in conjunction with EMA but without considering the parameter space stability loss, the model failed to achieve the expected performance improvement. Although TESS can capture the temporal dynamics of domain evolution, its adaptation process requires explicit guidance from parameter space constraints. Without $\mathcal{L}_{stab}$ to ensure smooth evolution, the controller may experience unstable state transitions, thereby impairing knowledge preservation. Therefore, the parameter space stability loss not only serves as a regularization mechanism to prevent forgetting but is also a key mechanism to ensure the coherent and effective evolution of TESS. This ablation study shows that the success of this method stems from the rational integration of its components, in which the TESS

Table 3: Adaptation time comparison between STEM and GCAL. We report the average time per domain adaptation and the corresponding speedup ratio.

Dataset	GCAL (s)	STEM (s)	Speedup
Twitch	7.241	0.794	9.12×
OGB	63.460	0.557	113.90×

controller achieves an effective balance between adaptability and stability through a carefully designed stabilization mechanism.

4.3.2 Hyperparameter Sensitivity. We further evaluate the robustness of hyperparameter selection. We primarily analyze the impact of two parameters: the EMA momentum coefficient α and the parameter stability loss weight λ_{stab} . The results are shown in Figure 6. Changes in the EMA momentum coefficient or the stability loss weight near their respective optimum values do not cause significant fluctuations in the average performance. On both datasets, the model maintains stable and excellent performance despite hyperparameter variations. This stability demonstrates the robustness of the method, indicating that its superior performance does not depend on fine-tuning of hyperparameters. Since exhaustive hyperparameter search can be very costly, this is crucial for practical deployment.

4.3.3 Computational Efficiency. We measured the average adaptation time for each domain. As shown in Table 3, method STEM achieves a significant speedup of 9.12x on Twitch and 113.90x on OGB. This superior efficiency stems from our lightweight adapter architecture and the absence of a replay mechanism, thus avoiding costly historical data storage and reprocessing. These results confirm that method STEM delivers excellent results with high efficiency, enabling our model to be practical in real-time deployment scenarios.

5 Conclusion

This paper addresses continual adaptation for graph neural networks on evolving domains by proposing a novel replay-free framework. Our core contribution is a paradigm shift from preserving historical knowledge to modeling temporal dynamics of parameter evolution. We introduce a controller-worker architecture where a lightweight state space controller dynamically generates adapter parameters for a frozen base GNN. Coupled with an unsupervised task objective and parameter space stability constraint, this approach achieves a robust balance between plasticity and stability, validated through theoretical analysis and extensive experiments. Future work may explore more advanced temporal models for complex long-range evolution patterns or adaptive domain summary mechanisms to enhance adaptation in non-stationary environments.

Acknowledgment

Ming Zhang and Junyu Luo are supported by grants from the National Key Research and Development Program of China with Grant No. 2023YFC3341203 and the National Natural Science Foundation of China (NSFC Grant Number 62276002).

References

- [1] Jie Cai, Xin Wang, Chaoyu Guan, Yateng Tang, Jin Xu, Bin Zhong, and Wenwu Zhu. 2022. Multimodal continual graph learning with neural architecture search. In *Proceedings of the ACM Web Conference 2022*. 1292–1300.
- [2] Rui Ding, Jielong Yang, Feng Ji, Xionghu Zhong, and Linbo Xie. 2024. FR-GNN: Mitigating the Impact of Distribution Shift On Graph Neural Networks via Test-Time Feature Reconstruction. *IEEE Internet of Things Journal* (2024).
- [3] Zhengming Ding, Sheng Li, Ming Shao, and Yun Fu. 2018. Graph adaptive knowledge transfer for unsupervised domain adaptation. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 37–52.
- [4] Zhihao Ding, Jieming Shi, Shiqi Shen, Xuequn Shang, Jiannong Cao, Zhipeng Wang, and Zhi Gong. 2024. Sgood: Substructure-enhanced graph-level out-of-distribution detection. In *Proc. of ACM International Conference on Information and Knowledge Management*.
- [5] Hao Dong, Zhiyuan Ning, Pengyang Wang, Ziyue Qiao, Pengfei Wang, Yuanchun Zhou, and Yanjie Fu. 2023. Adaptive path-memory network for temporal knowledge graph reasoning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. 2086–2094.
- [6] Gintare Karolina Dziugaite, Daniel M Roy, and Zoubin Ghahramani. 2015. Training generative neural networks via maximum mean discrepancy optimization. *arXiv preprint arXiv:1505.03906* (2015).
- [7] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario March, and Victor Lempitsky. 2016. Domain-adversarial training of neural networks. *Journal of machine learning research* 17, 59 (2016), 1–35.
- [8] Anchun Gui, Jinqiang Ye, and Han Xiao. 2024. G-adapter: Towards structure-aware parameter-efficient transfer learning for graph transformer networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 12226–12234.
- [9] Will Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [10] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608* (2024).
- [11] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [12] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*. PMLR, 2790–2799.
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [14] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [15] Wei Jin, Tong Zhao, Jiayuan Ding, Yozen Liu, Jiliang Tang, and Neil Shah. 2022. Empowering Graph Representation Learning with Test-Time Graph Transformation. In *The Eleventh International Conference on Learning Representations*.
- [16] Jeroen Kazius, Ross McGuire, and Roberta Bursi. 2005. Derivation and validation of toxicophores for mutagenicity prediction. In *Journal of medicinal chemistry*.
- [17] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [18] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [19] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190* (2021).
- [20] Yanghao Li, Naiyan Wang, Jianping Shi, Jiaying Liu, and Xiaodi Hou. 2016. Re-visiting batch normalization for practical domain adaptation. *arXiv preprint arXiv:1603.04779* (2016).
- [21] Huihui Liu, Yiding Yang, and Xinchao Wang. 2021. Overcoming catastrophic forgetting in graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 8653–8661.
- [22] Shikun Liu, Tianchun Li, Yongbin Feng, Nhan Tran, Han Zhao, Qiang Qiu, and Pan Li. 2023. Structural Re-weighting Improves Graph Domain Adaptation. In *International Conference on Machine Learning*. PMLR, 21778–21793.
- [23] Yilun Liu, Ruihong Qiu, and Zi Huang. 2023. Cat: Balanced continual graph learning with graph condensation. In *2023 IEEE International Conference on Data Mining (ICDM)*. IEEE, 1157–1162.
- [24] Chengqiang Lu, Qi Liu, Chao Wang, Zhenya Huang, Peize Lin, and Lixin He. 2019. Molecular property prediction: A multilevel quantum interactions modeling perspective. In *Proc. of Association for the Advancement of Artificial Intelligence*.
- [25] Junyu Luo, Yiyang Gu, Xiao Luo, Wei Ju, Zhiping Xiao, Yusheng Zhao, Jingyang Yuan, and Ming Zhang. 2024. GALA: Graph Diffusion-Based Alignment With Jigsaw for Source-Free Domain Adaptation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 12 (2024), 9038–9051.
- [26] Junyu Luo, Yuhao Tang, Yiwei Fu, Xiao Luo, Zhizhuo Kou, Zhiping Xiao, Wei Ju, Wentao Zhang, and Ming Zhang. 2025. Sparse Causal Discovery with Generative Intervention for Unsupervised Graph Domain Adaptation. In *Forty-second International Conference on Machine Learning*.
- [27] Junyu Luo, Zhiping Xiao, Yifan Wang, Xiao Luo, Jingyang Yuan, Wei Ju, Langechuan Liu, and Ming Zhang. 2024. Rank and Align: Towards Effective Source-free Graph Domain Adaptation. In *IJCAI*.
- [28] Xinhong Ma, Tianzhu Zhang, and Changsheng Xu. 2019. Gcan: Graph convolutional adversarial network for unsupervised domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8266–8276.
- [29] Chaoxi Niu, Guansong Pang, Ling Chen, and Bing Liu. [n. d.]. Replay-and-Forget-Free Graph Class-Incremental Learning: A Task Profiling and Prompting Approach. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [30] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Yaofo Chen, Shijian Zheng, Peilin Zhao, and Minghui Tan. 2022. Efficient test-time model adaptation without forgetting. In *International conference on machine learning*. PMLR, 16888–16905.
- [31] Francesco Orsini, Paolo Frasconi, and Luc De Raedt. 2015. Graph invariant kernels. In *Proc. of International Joint Conference on Artificial Intelligence*.
- [32] Ziyue Qiao, Qianyi Cai, Hao Dong, Jiawei Gu, Pengyang Wang, Meng Xiao, Xiao Luo, and Hui Xiong. 2025. GCAL: Adapting Graph Models to Evolving Domain Shifts. In *International Conference on Machine Learning*.
- [33] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2021. Multi-scale attributed node embedding. *Journal of Complex Networks* 9, 2 (2021), cnab014.
- [34] Li Sun, Junda Ye, Hao Peng, Feiyang Wang, and S Yu Philip. 2023. Self-supervised continual graph learning in adaptive riemannian spaces. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 4633–4642.
- [35] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [36] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. 2021. Tent: Fully Test-Time Adaptation by Entropy Minimization. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=uXl3bZLkr3c>
- [37] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. 2022. Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7201–7211.
- [38] Xiaoying Wang, Maoteng Duan, Jingxian Li, Anjun Ma, Gang Xin, Dong Xu, Zihai Li, Bingqiang Liu, and Qin Ma. 2024. MarsGT: Multi-omics analysis for rare population inference using single-cell graph transformer. *Nature Communications* 15, 1 (2024), 338.
- [39] Man Wu, Shirui Pan, and Xingquan Zhu. 2022. Attraction and Repulsion: Unsupervised Domain Adaptive Graph Contrastive Learning Network. *IEEE Transactions on Emerging Topics in Computational Intelligence* (2022).
- [40] Qitian Wu, Hengrui Zhang, Junchi Yan, and David Wipf. 2022. Handling distribution shifts on graphs: An invariance perspective. *International Conference on Learning Representations* (2022).
- [41] Wei Wu, Chao Wang, Dazhong Shen, Chuan Qin, Liyi Chen, and Hui Xiong. 2024. Afdgc: Adaptive feature de-correlation graph collaborative filtering for recommendations. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1242–1252.
- [42] Jiaren Xiao, Quanyu Dai, Xiaochen Xie, Qi Dou, Ka-Wai Kwok, and James Lam. 2022. Domain Adaptive Graph Infomax Via Conditional Adversarial Networks. *IEEE Transactions on Network Science and Engineering* (2022).
- [43] Yishi Xu, Yingxue Zhang, Wei Guo, Huifeng Guo, Ruiming Tang, and Mark Coates. 2020. Graphsail: Graph structure aware incremental learning for recommender systems. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2861–2868.
- [44] Ye Yuan, Chengwu Liu, Jingyang Yuan, Gongbo Sun, Siqi Li, and Ming Zhang. 2024. A hybrid RAG system with comprehensive enhancement on complex reasoning. *arXiv preprint arXiv:2408.05141* (2024).
- [45] Ye Yuan, Kexin Tang, Jianhao Shen, Ming Zhang, and Chenguang Wang. 2024. Measuring social norms of large language models. *arXiv preprint arXiv:2404.02491* (2024).
- [46] Peiyan Zhang, Yuchen Yan, Chaozhao Li, Senzhang Wang, Xing Xie, Guojie Song, and Sunghun Kim. 2023. Continual learning on dynamic graphs via parameter isolation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 601–611.
- [47] Xikun Zhang, Dongjin Song, Yixin Chen, and Dacheng Tao. 2024. Topology-aware Embedding Memory for Learning on Expanding Graphs. *arXiv preprint arXiv:2401.13200* (2024).
- [48] Xikun Zhang, Dongjin Song, and Dacheng Tao. 2022. Hierarchical prototype networks for continual graph representation learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 4 (2022), 4622–4636.
- [49] Xikun Zhang, Dongjin Song, and Dacheng Tao. 2022. Sparsified subgraph memory for continual graph representation learning. In *2022 IEEE International Conference*

- on Data Mining (ICDM). IEEE, 1335–1340.
- [50] Xikun Zhang, Dongjin Song, and Dacheng Tao. 2023. Ricci curvature-based graph sparsification for continual graph representation learning. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
- [51] Chenguang Zheng, Hongzhi Chen, Yuxuan Cheng, Zhezhong Song, Yifan Wu, Changji Li, James Cheng, Hao Yang, and Shuai Zhang. 2022. ByteGNN: efficient graph neural network training at large scale. *Proceedings of the VLDB Endowment* 15, 6 (2022), 1228–1242.
- [52] Fan Zhou and Chengtai Cao. 2021. Overcoming catastrophic forgetting in graph neural networks with experience replay. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 4714–4722.

A Proofs of Theoretical Results

This appendix provides detailed proofs for the theoretical guarantees presented in Section 3. We formalize the notation, state our assumptions, and then rigorously derive the bounds on catastrophic forgetting (Theorem 3.1) and the convergence stability of the adaptation process (Theorem 3.2).

To formalize the proof, we introduce the following notation:

Let $\theta \in \mathbb{R}^p$ represent the collection of all trainable parameters of the TESS controller and the Parameter Generator, i.e., $\theta = (\Theta_{TESS}, \Theta_{gen})$.

Let $\Phi_t : \mathbb{R}^p \rightarrow \mathbb{R}^m$ be the map from the controller parameters θ to the vectorized adapter parameters $\text{vec}(\Delta\Theta_t)$. This function encapsulates the entire process from the domain summary $\hat{h}_t$ through the TESS and the generator.

Let $a_{t-1} := \text{vec}(\Delta\Theta_{t-1})$ denote the vectorized adapter parameters from the previous time step, which is treated as a constant during optimization at time step t .

The Worker GNN’s output on a graph G with adapter $\Delta\Theta$ is denoted by the vector $f(G; \Theta_{base} \oplus \Delta\Theta) \in \mathbb{R}^d$. The stability loss can now be written compactly as $\mathcal{L}_{stab}(\theta) = \|\Phi_t(\theta) - a_{t-1}\|_2^2$.

Our proofs rely on the following standard assumptions.

A1: For any fixed graph G , the GNN output function $f(G; \cdot)$ is L -Lipschitz continuous with respect to its vectorized adapter parameters. For any two adapter vectors $a, a' \in \mathbb{R}^m$:

$$\|f(G; \Theta_{base} \oplus \text{reshape}(a)) - f(G; \Theta_{base} \oplus \text{reshape}(a'))\|_2 \leq L \cdot \|a - a'\|_2. \quad (15)$$

This is a mild assumption, as neural networks composed of common layers and activations are Lipschitz continuous.

A2: For the optimization at time step t , the task loss $\mathcal{L}_{task}(\theta)$ is convex with respect to the controller parameters θ . While deep learning objectives are generally non-convex, this is a common idealization for theoretical analysis that focuses on the regularizing effect of a specific loss term.

A3: The map $\Phi_t(\theta)$ is twice continuously differentiable (C^2). Its Jacobian is denoted $J_t(\theta) = \frac{\partial \Phi_t(\theta)}{\partial \theta} \in \mathbb{R}^{m \times p}$. This holds true as the TESS and generator are composed of standard neural network layers.

A.1 Proof of Theorem 1 (Bound on Catastrophic Forgetting)

THEOREM A.1 (BOUND ON CATASTROPHIC FORGETTING). *Assume the Worker GNN’s output function $f(G; \Theta_{base} \oplus \Delta\Theta)$ is L -Lipschitz continuous with respect to its adapter parameters $\Delta\Theta$. Then, when transitioning from domain $t-1$ to domain t , the expected change in the model’s outputs on all historical graph domains $G_{<t} = \{G_1, \dots, G_{t-1}\}$*

is bounded by the parameter evolution stability loss:

$$\mathbb{E}_{G \sim G_{<t}} [\|f(G; \Theta_{base} \oplus \Delta\Theta_t) - f(G; \Theta_{base} \oplus \Delta\Theta_{t-1})\|_2] \leq L \cdot \|\Delta\Theta_t - \Delta\Theta_{t-1}\|_F = L \cdot \sqrt{\mathcal{L}_{stab}}, \quad (16)$$

where L is the Lipschitz constant and $\|\cdot\|_F$ is the Frobenius norm.

Proof. The proof is a direct application of the Lipschitz assumption.

For any single historical graph $G \in G_{<t}$, we apply the Lipschitz continuity from A1 with $a = \text{vec}(\Delta\Theta_t) = \Phi_t(\theta)$ and $a' = \text{vec}(\Delta\Theta_{t-1}) = a_{t-1}$. This gives:

$$\|f(G; \dots \oplus \Delta\Theta_t) - f(G; \dots \oplus \Delta\Theta_{t-1})\|_2 \leq L \cdot \|\text{vec}(\Delta\Theta_t) - \text{vec}(\Delta\Theta_{t-1})\|_2. \quad (17)$$

We take the expectation of both sides over the distribution of historical graphs $G \sim G_{<t}$. Since the right-hand side of the inequality does not depend on G , it remains unchanged:

$$\mathbb{E}_{G \sim G_{<t}} [\|f(\dots)\|_2] \leq L \cdot \|\text{vec}(\Delta\Theta_t) - \text{vec}(\Delta\Theta_{t-1})\|_2. \quad (18)$$

By definition, $\mathcal{L}_{stab} = \|\text{vec}(\Delta\Theta_t) - \text{vec}(\Delta\Theta_{t-1})\|_2^2$. Therefore, the L2-norm term is exactly $\sqrt{\mathcal{L}_{stab}}$. Substituting this in yields the final result:

$$\mathbb{E}_{G \sim G_{<t}} [\|f(\dots)\|_2] \leq L \cdot \sqrt{\mathcal{L}_{stab}}. \quad (19)$$

This completes the proof. By minimizing $\mathcal{L}_{stab}$, we are directly minimizing a formal upper bound on functional forgetting.

A.2 Proof of Theorem 2 (Convergence and Stability)

THEOREM A.2 (CONVERGENCE AND STABILITY OF THE ADAPTATION PROCESS). *Assume that for any domain t , the task loss $\mathcal{L}_{task}$ is convex with respect to the controller parameters $(\Theta_{TESS}, \Theta_{gen})$ and has bounded gradients. Under the regularization imposed by the parameter evolution stability loss $\mathcal{L}_{stab}$, the total loss function $\mathcal{L}_{total}$ becomes locally strongly convex near a minimizer under standard assumptions. Consequently, the adaptation optimization process on each domain converges to a unique optimal controller parameter, and this solution is stable. Formally, when the Jacobian of the adapter generation map has full rank and the residual is sufficiently small, the Hessian of the total loss satisfies:*

$$\nabla^2 \mathcal{L}_{total} = \nabla^2 \mathcal{L}_{task} + \lambda_{stab} \nabla^2 \mathcal{L}_{stab} \succ 0, \quad (20)$$

implying that the optimization problem is locally strongly convex near the solution.

Discussion. The original claim of strong convexity must be refined because $\mathcal{L}_{stab}(\theta) = \|\Phi_t(\theta) - a_{t-1}\|_2^2$ is the squared norm of a non-linear function of θ . While such a term provides regularization, its ability to induce strong convexity depends on the local properties of the map Φ_t . The following proof makes this precise.

Proof.

The total loss is

$$\mathcal{L}_{total}(\theta) = \mathcal{L}_{task}(\theta) + \lambda_{stab} \mathcal{L}_{stab}(\theta). \quad (21)$$

Its Hessian matrix is:

$$\nabla^2 \mathcal{L}_{total}(\theta) = \nabla^2 \mathcal{L}_{task}(\theta) + \lambda_{stab} \nabla^2 \mathcal{L}_{stab}(\theta). \quad (22)$$

By Assumption A2, the term $\mathcal{L}_{\text{task}}$ is convex, meaning its Hessian is positive semidefinite (PSD)

$$\nabla^2 \mathcal{L}_{\text{task}} \succeq 0. \quad (23)$$

We now analyze the Hessian of the stability loss, $\mathcal{L}_{\text{stab}}(\theta) = \|\Phi_t(\theta) - a_{t-1}\|_2^2$. Let $r_t(\theta) = \Phi_t(\theta) - a_{t-1}$ be the residual vector. Using the chain rule, the Hessian of $\mathcal{L}_{\text{stab}}$ is:

$$\nabla^2 \mathcal{L}_{\text{stab}}(\theta) = 2J_t(\theta)^T J_t(\theta) + 2 \sum_{k=1}^m r_{t,k}(\theta) \nabla^2 \Phi_{t,k}(\theta). \quad (24)$$

where $J_t(\theta)$ is the Jacobian of Φ_t , and $r_{t,k}$ and $\Phi_{t,k}$ are the k -th components of the residual and map, respectively.

The first term, $2J_t(\theta)^T J_t(\theta)$, is the Gauss-Newton approximation of the Hessian. This matrix is always positive semidefinite. If the Jacobian $J_t(\theta)$ has full column rank (i.e., the columns are linearly independent), then $J_t(\theta)^T J_t(\theta)$ is positive definite (PD). Let its smallest eigenvalue be $2s_0^2 > 0$.

The second term involves second derivatives of the map Φ_t . In continual adaptation, we expect the new adapter parameters $\Phi_t(\theta)$ to be a small perturbation of the previous ones a_{t-1} . This means the residual vector $r_t(\theta)$ will be small near the solution. The contribution of this second term is bounded by the magnitude of the residual. Specifically, its effect on convexity is bounded by $2\|r_t(\theta)\|_2 \beta_\Phi$, where β_Φ is an upper bound on the operator norm of the Hessian components $\nabla^2 \Phi_{t,k}(\theta)$.

Combining these, the Hessian of $\mathcal{L}_{\text{total}}$ satisfies:

$$\begin{aligned} \nabla^2 \mathcal{L}_{\text{total}}(\theta) &\succeq \lambda_{\text{stab}} \nabla^2 \mathcal{L}_{\text{stab}}(\theta) \\ &\succeq \lambda_{\text{stab}} \left(2J_t(\theta)^T J_t(\theta) - 2\|r_t(\theta)\|_2 \beta_\Phi I \right). \end{aligned} \quad (25)$$

If we are in a neighborhood of a solution where $J_t(\theta)$ has a minimum singular value of $s_0 > 0$ and the residual is small enough such that $s_0^2 > \|r_t(\theta)\|_2 \beta_\Phi$, then the right-hand side is strictly positive definite.

The total loss $\mathcal{L}_{\text{total}}$ becomes locally strongly convex. The term $2\lambda_{\text{stab}} J_t^T J_t$ from the stability loss dominates the other terms, ensuring the Hessian is positive definite. A function that is locally strongly convex around a minimizer admits a unique local solution, and gradient-based approaches are guaranteed to converge linearly to it.

This proves that $\mathcal{L}_{\text{stab}}$ acts as a powerful regularizer that stabilizes the optimization landscape, ensuring that each adaptation step is principled and convergent.

B Additional Experimental Details

B.1 Implementation Details

For the Worker GNN backbone architecture, we adopt a two-layer Graph Convolutional Network (GCN) [18] across all datasets, with the exception of OGB-Arxiv, where we employ GraphSAGE [9] to better handle the large-scale citation network structure. During the source domain pretraining phase, the training, validation, and test splits are configured as 60%, 20%, and 20%, respectively. All models are optimized using the Adam optimizer [17] with a learning rate of 10^{-4} for Twitch-explicit, OGB-Arxiv, and FRANKENSTEIN, while a smaller learning rate of 10^{-5} is employed for Mutagenicity to ensure stable convergence. The weight decay coefficient is

uniformly set to 5×10^{-4} across all experiments. The pretraining phase typically runs for 100–200 epochs until convergence on the source domain, whereas the adaptation phase at each subsequent domain is performed for 1–10 epochs to balance adaptation effectiveness and computational efficiency. Comprehensive hyperparameter configurations and implementation details are provided in the accompanying source code for reproducibility.

For performance evaluation, we construct an accuracy matrix $M^{\text{result}} \in \mathbb{R}^{T \times T}$, which is lower-triangular, where $M_{i,j}^{\text{result}}$ (for $i \geq j$) denotes the model's performance on domain j after being trained up to domain i . Following established protocols [15], we measure performance using ROC-AUC for the Twitch-explicit dataset and classification accuracy for all other datasets. To provide aggregate performance metrics across the evolving domain sequence, we compute Average Performance (AP) as $\text{AP} = \frac{1}{T} \sum_{i=1}^T M_{T,i}^{\text{result}}$, which reflects the final model's retention of knowledge across all historical domains, and Evolving Average Performance (EAP) as $\text{EAP} = \frac{1}{T} \sum_{i=1}^T M_{i,i}^{\text{result}}$, which captures the model's immediate adaptation quality on each newly encountered domain. Each experimental configuration is repeated five times with different random seeds, and we report both the mean and standard deviation to ensure statistical reliability.